

# PlacementTrackerPro

## *Full Technical Reference Document*

---

A comprehensive campus placement preparation platform  
for engineering students targeting top tech companies in India

# Table of Contents

- Table of Contents .....2
- 1. Project Overview .....4
  - 1.1 Problem Statement .....4
  - 1.2 Key Value Propositions .....4
  - 1.3 Target Users.....4
- 2. Technology Stack .....5
  - 2.1 Frontend .....5
  - 2.2 Backend .....5
  - 2.3 AI Provider .....5
  - 2.4 Development Tooling .....6
- 3. System Architecture .....7
  - 3.1 High-Level Architecture .....7
  - 3.2 Backend Layer Structure.....7
  - 3.3 Frontend Layer Structure .....7
  - 3.4 Request-Response Flow .....8
- 4. Database Schema .....9
  - 4.1 User.....9
  - 4.2 Task.....9
  - 4.3 DSAProblem.....9
  - 4.4 CoreCSProgress..... 10
  - 4.5 PlacementApplication ..... 10
  - 4.6 InterviewQuestion..... 10
  - 4.7 Other Models..... 10
- 5. API Endpoints..... 12
  - 5.1 Authentication — /api/auth ..... 12
  - 5.2 Tasks — /api/tasks..... 12
  - 5.3 DSA Tracker — /api/dsa ..... 12
  - 5.4 Placement Applications — /api/placement-applications..... 12
  - 5.5 AI — /api/ai (all protected) ..... 13
  - 5.6 Other Routes ..... 13
- 6. Frontend Pages..... 14
- 7. AI Features (Groq Integration) ..... 15
  - 7.1 AI Service Architecture ..... 15
  - 7.2 AI Chat Assistant..... 15
  - 7.3 AI Daily Planner ..... 15

|                                                        |    |
|--------------------------------------------------------|----|
| 7.4 Company-Specific Interview Questions.....          | 15 |
| 7.5 ATS Resume Scorer .....                            | 15 |
| 7.6 Mock Interview .....                               | 15 |
| 7.7 Error Handling.....                                | 16 |
| 8. Authentication System .....                         | 17 |
| 8.1 Flow .....                                         | 17 |
| 8.2 JWT Details .....                                  | 17 |
| 8.3 Protected Routes.....                              | 17 |
| 9. Environment & Setup .....                           | 18 |
| 9.1 Prerequisites.....                                 | 18 |
| 9.2 Backend Environment Variables (backend/.env) ..... | 18 |
| 9.3 Start Commands .....                               | 18 |
| 9.4 Database Setup.....                                | 18 |
| 9.5 Useful Scripts .....                               | 18 |
| 10. UI Design System .....                             | 20 |
| 10.1 Theme.....                                        | 20 |
| 10.2 Tailwind Custom Tokens (tailwind.config.ts).....  | 20 |
| 10.3 Sidebar Navigation Groups.....                    | 20 |
| 10.4 Key Reusable Components .....                     | 20 |
| 11. Live Hackathon Feed.....                           | 22 |
| 12. Known Limitations & Future Improvements .....      | 23 |
| 12.1 Current Limitations .....                         | 23 |
| 12.2 Planned Improvements .....                        | 23 |
| 13. Full File Structure.....                           | 24 |

# 1. Project Overview

---

PlacementTrackerPro is a full-stack web application designed to help engineering students systematically prepare for campus placements. It combines structured learning trackers, AI-powered tools, placement application management, and contest discovery in a single, unified platform.

## 1.1 Problem Statement

Engineering students preparing for placements struggle to track progress across DSA, core CS subjects, projects, resume quality, interview prep, and company applications in a fragmented way. PlacementTrackerPro centralises all of this with data-backed dashboards and AI guidance.

## 1.2 Key Value Propositions

- Unified dashboard showing DSA progress, study sessions, tasks, and placement pipeline
- AI assistant powered by Groq LLaMA 3.3 70B — answers doubts, generates daily plans, creates company-specific interview questions
- ATS Resume Scorer — paste any resume and get an ATS compatibility score with breakdown
- Mock Interview — timed AI Q&A with per-answer evaluation and final percentage score
- Live Hackathon Feed — curated events (SIH, HackWithInfy, TCS CodeVita, Flipkart Grid, Google Hash Code, etc.)
- Placement Pipeline — track companies from Wishlist → Selected with round-by-round status

## 1.3 Target Users

- Final-year and pre-final-year engineering students (B.E. / B.Tech)
- Students targeting product companies, service companies, and startups
- Students participating in competitive programming contests and hackathons

## 2. Technology Stack

### 2.1 Frontend

| Technology      | Version | Purpose                            |
|-----------------|---------|------------------------------------|
| React           | 18.3.1  | UI component framework             |
| TypeScript      | 5.7.2   | Type-safe JavaScript               |
| Vite            | 6.0.5   | Build tool and dev server          |
| Tailwind CSS    | 3.4.17  | Utility-first styling              |
| React Router v6 | 6.28.1  | Client-side routing                |
| Axios           | 1.7.9   | HTTP client for API calls          |
| Lucide React    | 1.17.0  | Icon library                       |
| Recharts        | 3.8.1   | Charts and analytics visualisation |

### 2.2 Backend

| Technology        | Version | Purpose                                 |
|-------------------|---------|-----------------------------------------|
| Node.js + Express | 4.21.2  | HTTP server and REST API                |
| TypeScript        | 5.7.2   | Type-safe server code                   |
| tsx               | 4.19.2  | Dev-mode TypeScript runner (watch mode) |
| Prisma ORM        | 6.1.0   | Type-safe DB access and migrations      |
| PostgreSQL        | —       | Relational database                     |
| Zod               | 3.24.1  | Runtime schema validation               |
| jsonwebtoken      | 9.0.2   | JWT-based authentication                |
| bcrypt            | 5.1.1   | Password hashing                        |
| groq-sdk          | 1.3.0   | Groq AI API client (LLaMA models)       |
| dotenv            | 16.4.7  | Environment variable loading            |

### 2.3 AI Provider

**Provider:** Groq (console.groq.com) — free tier, extremely fast inference

**Primary Model:** llama-3.3-70b-versatile (best quality)

Fallback chain (tried in order if a model is unavailable or decommissioned):

- llama-3.3-70b-versatile
- llama-3.1-8b-instant
- llama3-70b-8192
- llama3-8b-8192
- mixtral-8x7b-32768

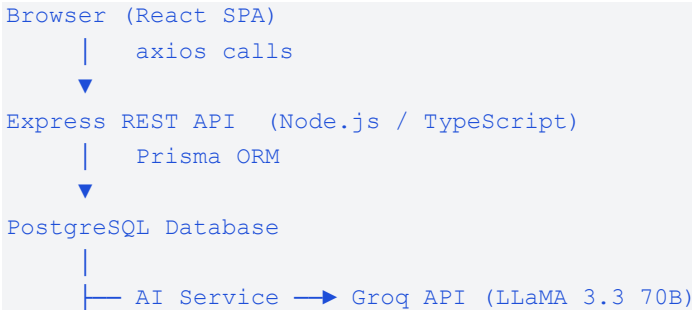
## 2.4 Development Tooling

- ESLint + typescript-eslint — linting
- Prisma Studio — database GUI (npx prisma studio)
- Vite HMR — instant frontend hot reload
- tsx --watch — backend hot reload in dev

### 3. System Architecture

#### 3.1 High-Level Architecture

PlacementTrackerPro follows a classic client–server architecture with a clear separation of frontend and backend concerns:



#### 3.2 Backend Layer Structure

The backend follows a strict layered architecture inside backend/src/:

| Layer         | Folder       | Responsibility                                          |
|---------------|--------------|---------------------------------------------------------|
| Config        | config/      | Env variable parsing and validation via Zod             |
| Routes        | routes/      | URL mapping — one file per domain (e.g. ai.routes.ts)   |
| Controllers   | controllers/ | Request parsing, input validation, response shaping     |
| Services      | services/    | Business logic, DB queries, AI calls                    |
| Middleware    | middleware/  | Auth guard, async error wrapper, request validator      |
| Validators    | validators/  | Zod schemas for each request body                       |
| Prisma Client | prisma/      | Single shared Prisma instance                           |
| Utils         | utils/       | JWT, password, AppError, date helpers                   |
| Types         | types/       | TypeScript interfaces for auth and express augmentation |

#### 3.3 Frontend Layer Structure

The frontend lives in frontend/src/ and is organized as follows:

| Folder      | Contents                                                                     |
|-------------|------------------------------------------------------------------------------|
| pages/      | 17 page components — one per route                                           |
| components/ | Reusable UI: common/, analytics/, tasks/, calendar/, forms/, learning/       |
| layouts/    | AppLayout.tsx — dark sidebar + topbar shell wrapping all authenticated pages |
| services/   | 13 service files — one per domain, wrapping axios calls                      |
| hooks/      | useAuth, useToast, useNotifications                                          |

| Folder    | Contents                                                     |
|-----------|--------------------------------------------------------------|
| contexts/ | AuthContext (JWT + user state), ToastContext (notifications) |
| types/    | TypeScript interfaces matching backend models                |
| utils/    | constants, date formatting, learning labels, env             |

### 3.4 Request-Response Flow

1. User action in the browser triggers a service call (e.g. `aiService.chat("explain BFS")`)
2. Axios interceptor injects Bearer token from `localStorage`
3. Express routes match → `asyncHandler` catches async errors automatically
4. `requireAuth` middleware verifies JWT and attaches user to request
5. Controller validates input and calls the service layer
6. Service performs DB queries (Prisma) or AI calls (Groq SDK)
7. Structured JSON response: `{ data: ... }` on success, `{ error: { message, code } }` on failure
8. Frontend catches errors via `getErrorMessage()` and shows toast/error state



## 4. Database Schema

Database: PostgreSQL. ORM: Prisma. All tables use UUID primary keys and cascade-delete on user removal.

### 4.1 User

| Field                 | Type     | Notes                       |
|-----------------------|----------|-----------------------------|
| id                    | UUID     | Primary key, auto-generated |
| name                  | String   | Full name                   |
| email                 | String   | Unique, used for login      |
| passwordHash          | String   | bcrypt hash (12 rounds)     |
| role                  | UserRole | STUDENT   ADMIN             |
| createdAt / updatedAt | DateTime | Auto-managed                |

### 4.2 Task

| Field                   | Type               | Notes                                 |
|-------------------------|--------------------|---------------------------------------|
| title / description     | String             | Task name and optional details        |
| category                | TaskCategory       | DSA   CORE_CS   FULL_STACK   APTITUDE |
| dueDate                 | DateTime           | Used in calendar view                 |
| priority                | TaskPriority       | HIGH   MEDIUM   LOW                   |
| status                  | TaskStatus         | PENDING   IN_PROGRESS   COMPLETED     |
| completed / completedAt | Boolean / DateTime | Completion tracking                   |

### 4.3 DSAProblem

| Field                        | Type             | Notes                              |
|------------------------------|------------------|------------------------------------|
| title / topic                | String           | Problem name and DSA topic         |
| difficulty                   | DsaDifficulty    | EASY   MEDIUM   HARD               |
| status                       | DsaProblemStatus | SOLVED   PENDING   REVISION_NEEDED |
| leetcodeLink / youtubeLink   | String?          | Optional resource links            |
| notes                        | String?          | Personal notes                     |
| revisionCount / lastSolvedAt | Int / DateTime?  | Spaced repetition support          |

## 4.4 CoreCSProgress

| Field   | Type           | Notes                                                   |
|---------|----------------|---------------------------------------------------------|
| subject | CoreCSSubject  | OS   DBMS   CN   OOP                                    |
| topic   | String         | Specific topic within subject                           |
| status  | LearningStatus | NOT_STARTED   IN_PROGRESS   COMPLETED   REVISION_NEEDED |
| notes   | String?        | User notes                                              |

## 4.5 PlacementApplication

| Field                          | Type            | Notes                                                                                                          |
|--------------------------------|-----------------|----------------------------------------------------------------------------------------------------------------|
| companyName / role             | String          | Company and job title                                                                                          |
| packageLPA                     | Float           | Offered CTC in LPA                                                                                             |
| status                         | PlacementStatus | WISHLIST → APPLIED → OA_SCHEDULED → OA_CLEARED → INTERVIEW_SCHEDULED → INTERVIEW_CLEARED → SELECTED   REJECTED |
| round                          | PlacementRound  | NONE   OA   TECHNICAL_1/2   MANAGERIAL   HR   FINAL                                                            |
| applicationDate / deadlineDate | DateTime        | Timeline tracking                                                                                              |
| jobLink / notes                | String?         | Optional extras                                                                                                |

## 4.6 InterviewQuestion

| Field                   | Type                    | Notes                                                                                                |
|-------------------------|-------------------------|------------------------------------------------------------------------------------------------------|
| question                | String                  | The interview question text                                                                          |
| category                | InterviewCategory       | HR_INTERVIEW   BEHAVIORAL   DSA_INTERVIEW   CORE_CS_INTERVIEW   SYSTEM_DESIGN   FULL_STACK_INTERVIEW |
| difficulty              | InterviewDifficulty     | EASY   MEDIUM   HARD                                                                                 |
| answerNotes / userNotes | String?                 | AI-generated hints + personal notes                                                                  |
| status                  | InterviewQuestionStatus | NOT_STARTED   PRACTICED   MASTERED                                                                   |

## 4.7 Other Models

| Model             | Key Fields                                                                 |
|-------------------|----------------------------------------------------------------------------|
| StudyPreference   | studyMode: WEEKDAYS_ONLY   ALTERNATE_DAYS   WEEKENDS_ONLY                  |
| FullStackProgress | technology: REACT   ANGULAR   SPRING_BOOT   DOTNET, status: LearningStatus |

| Model         | Key Fields                                                                                                      |
|---------------|-----------------------------------------------------------------------------------------------------------------|
| StudySession  | durationMinutes, category: DSA   CORE_CS   FULL_STACK   APTITUDE, notes                                         |
| MockInterview | company, interviewType, date, rating (1-10), feedback, strengths, weaknesses                                    |
| ResumeVersion | resumeName, version, targetRole, checklist flags (6 fields), ATS flags (5 fields)                               |
| ExamDate      | title, date, type: PLACEMENT   EXAM   INTERVIEW   CONTEST, notes                                                |
| Opportunity   | title, organizer, type: HACKATHON   INTERNSHIP   CONTEST   FELLOWSHIP, registrationDeadline, link, isRegistered |

## 5. API Endpoints

All endpoints are prefixed with /api. Protected routes require Authorization: Bearer <jwt\_token>. All responses use { data: ... } or { error: { message, code } }.

### 5.1 Authentication — /api/auth

| Method | Path    | Auth | Description                                 |
|--------|---------|------|---------------------------------------------|
| POST   | /signup | No   | Register a new user (name, email, password) |
| POST   | /login  | No   | Login, returns JWT access token             |
| GET    | /me     | Yes  | Get current user profile                    |

### 5.2 Tasks — /api/tasks

| Method | Path       | Description                                           |
|--------|------------|-------------------------------------------------------|
| GET    | /          | List all tasks (filterable by category, status, date) |
| POST   | /          | Create a task                                         |
| PATCH  | /:id       | Update a task (including mark complete)               |
| DELETE | /:id       | Delete a task                                         |
| GET    | /analytics | Task completion stats for dashboard                   |

### 5.3 DSA Tracker — /api/dsa

| Method | Path              | Description                                            |
|--------|-------------------|--------------------------------------------------------|
| GET    | /                 | List problems with filters (topic, difficulty, status) |
| POST   | /                 | Add a new problem                                      |
| PATCH  | /:id              | Update status, notes, revision count                   |
| DELETE | /:id              | Remove problem                                         |
| GET    | /seed-neetcode150 | Seed 150 NeetCode problems into user account           |
| GET    | /analytics        | Topic-wise and difficulty-wise breakdown               |

### 5.4 Placement Applications — /api/placement-applications

| Method | Path | Description                        |
|--------|------|------------------------------------|
| GET    | /    | List all applications with filters |
| POST   | /    | Add a company application          |
| PATCH  | /:id | Update status, round, notes        |

| Method | Path        | Description              |
|--------|-------------|--------------------------|
| DELETE | /:id        | Remove application       |
| GET    | /statistics | Summary counts by status |

## 5.5 AI — /api/ai (all protected)

| Method | Endpoint             | Input                                 | Output                                                                                      |
|--------|----------------------|---------------------------------------|---------------------------------------------------------------------------------------------|
| POST   | /chat                | { message }                           | AI text reply (placement prep assistant)                                                    |
| POST   | /daily-plan          | { }                                   | Personalised study plan based on user's DB data                                             |
| POST   | /interview-questions | { company, category }                 | Array of 8 { question, difficulty, notes }                                                  |
| POST   | /score-resume        | { resumeText }                        | { score, breakdown:{keywords,format,sections,impact,brevity}, strengths[], improvements[] } |
| POST   | /evaluate-answer     | { question, answer, category }        | { score, feedback, strengths[], improvements[] }                                            |
| POST   | /mock-question       | { company, category, questionNumber } | Single question string                                                                      |

## 5.6 Other Routes

| Route Prefix        | Key Endpoints                                                                  |
|---------------------|--------------------------------------------------------------------------------|
| /api/core-cs        | GET / • POST / • PATCH /:id — Core CS topic progress tracking                  |
| /api/full-stack     | GET / • PATCH /:id — Full-stack technology progress                            |
| /api/study-sessions | GET / • POST / • DELETE /:id — Study session logging                           |
| /api/analytics      | GET / — Combined learning analytics across DSA + CoreCS + FullStack            |
| /api/interview-prep | GET / • POST / • PATCH /:id • GET /statistics — Question bank CRUD             |
| /api/resumes        | GET / • POST / • GET /statistics — Resume version management                   |
| /api/exam-dates     | GET / • POST / • PATCH /:id • DELETE /:id — Important date tracking            |
| /api/opportunities  | GET / • POST / • PATCH /:id • DELETE /:id • GET /live-feed — Hackathon tracker |
| /api/preferences    | GET / • PUT / — Study preference (weekday mode)                                |
| /api/health         | GET / — Health check endpoint                                                  |

## 6. Frontend Pages

All pages are protected by `ProtectedRoute` and wrapped in `AppLayout` (dark sidebar + topbar). Public routes: `/login`, `/signup`.

| Route                           | Page                              | Description                                                                                                                       |
|---------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>/dashboard</code>         | <code>DashboardPage</code>        | Stats overview: task completion ring, DSA progress bars, placement pipeline summary, upcoming interviews, AI daily plan generator |
| <code>/tasks</code>             | <code>TasksPage</code>            | Full task CRUD with category/priority/date filters                                                                                |
| <code>/calendar</code>          | <code>CalendarPage</code>         | Month-view calendar showing tasks and exam dates                                                                                  |
| <code>/dsa-tracker</code>       | <code>DSATrackerPage</code>       | DSA problem list with status tracking, NeetCode 150 seed, topic/difficulty filters                                                |
| <code>/core-cs</code>           | <code>CoreCSPage</code>           | Progress tracking for OS, DBMS, CN, OOP topics                                                                                    |
| <code>/full-stack</code>        | <code>FullStackPage</code>        | Technology progress: React, Angular, Spring Boot, .NET                                                                            |
| <code>/study-history</code>     | <code>StudyHistoryPage</code>     | Log and review study sessions by date and category                                                                                |
| <code>/analytics</code>         | <code>AnalyticsPage</code>        | Charts: study time by week, topic breakdown, progress trends                                                                      |
| <code>/placement-tracker</code> | <code>PlacementTrackerPage</code> | Kanban-style pipeline: Wishlist → Selected, filter by company/status                                                              |
| <code>/interview-prep</code>    | <code>InterviewPrepPage</code>    | 3 tabs: AI Questions (company-specific), Question Bank CRUD, Mock Interviews log                                                  |
| <code>/mock-interview</code>    | <code>MockInterviewPage</code>    | Timed AI interview: 2-min countdown per question, real-time answer evaluation, final score                                        |
| <code>/resume-hub</code>        | <code>ResumeHubPage</code>        | 2 tabs: ATS Scorer (paste & score), Resume Versions (track multiple resume files)                                                 |
| <code>/hackathons</code>        | <code>HackathonsPage</code>       | 2 tabs: Live Feed (10 curated events with days-left countdown), My List (saved opportunities)                                     |
| <code>/ai-assistant</code>      | <code>AIAssistantPage</code>      | Chat interface with Groq LLaMA — placement prep Q&A, DSA help, HR advice                                                          |
| <code>/settings</code>          | <code>SettingsPage</code>         | Profile info, study mode preference, theme toggle                                                                                 |

## 7. AI Features (Groq Integration)

---

### 7.1 AI Service Architecture

File: backend/src/services/ai.service.ts

All AI calls go through the callGroq() internal function which tries models in order, skipping any that return 404, "not found", "decommissioned", or 429-with-limit-zero errors.

### 7.2 AI Chat Assistant

**Endpoint:** POST /api/ai/chat

A placement-specialist LLM assistant. The system prompt instructs the model to focus on: DSA (with time/space complexity), Core CS subjects (OS/DBMS/CN/OOP), HR interview answers, system design basics, resume tips, and study roadmaps.

### 7.3 AI Daily Planner

**Endpoint:** POST /api/ai/daily-plan

Before calling Groq, the service queries Prisma in parallel for:

- DSA problems with status REVISION\_NEEDED
- Core CS topics with status NOT\_STARTED (up to 5)
- Tasks due today
- Tasks due within next 3 days

This contextual data is injected into the prompt, producing a personalised time-blocked study plan.

### 7.4 Company-Specific Interview Questions

**Endpoint:** POST /api/ai/interview-questions

Inputs: company name (e.g. "Google"), category (e.g. "DSA\_INTERVIEW"). The model returns a JSON array of 8 questions with difficulty (EASY/MEDIUM/HARD) and answer notes, parsed with a regex JSON extractor.

### 7.5 ATS Resume Scorer

**Endpoint:** POST /api/ai/score-resume

Paste up to 6000 characters of resume text. The AI acts as an ATS evaluator and returns JSON with:

- Overall score (0–100)
- 5-category breakdown: Keywords, Format, Sections, Impact, Brevity
- Strength points (array)
- Improvement suggestions (array)

### 7.6 Mock Interview

**Question generation:** POST /api/ai/mock-question

**Answer evaluation:** POST /api/ai/evaluate-answer

The mock interview flow in MockInterviewPage.tsx:

9. User selects company + category → AI generates first question
10. 2-minute countdown per question (auto-submits on timeout)
11. Answer submitted → AI evaluates: score (1-10), feedback, strengths, improvements
12. Next question is pre-fetched in background while user reads feedback
13. After all questions: final score shown as percentage with per-question breakdown

## 7.7 Error Handling

AppError class carries statusCode and code. All AI errors surface specific codes: GROQ\_KEY\_MISSING (500), GROQ\_API\_ERROR (502), GROQ\_MODEL\_UNAVAILABLE (502), AI\_PARSE\_ERROR (502). The frontend shows the exact server error message via getErrorMessage().



## 8. Authentication System

---

### 8.1 Flow

14. POST /api/auth/signup — password hashed with bcrypt (12 rounds), user stored in DB
15. POST /api/auth/login — password verified, JWT signed with HS256 (7-day expiry)
16. Frontend stores token in localStorage, Axios interceptor injects it as Bearer token
17. requireAuth middleware verifies token signature, then queries DB to confirm user still exists
18. request.user is set to { id, email, name, role } for downstream use

### 8.2 JWT Details

- Algorithm: HS256
- Expiry: 7 days (configurable via JWT\_EXPIRES\_IN env var)
- Payload: { sub: userId, email, name, role, iat, exp }

### 8.3 Protected Routes

All routes except /api/auth/\* and /api/health are protected. The asyncHandler(requireAuth) pattern wraps the middleware to catch async errors cleanly.

## 9. Environment & Setup

### 9.1 Prerequisites

- Node.js 18+ and npm
- PostgreSQL 15+ running locally
- A Groq API key from console.groq.com (free account)

### 9.2 Backend Environment Variables (backend/.env)

| Variable           | Example / Notes                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| NODE_ENV           | development                                                                       |
| PORT               | 5000                                                                              |
| DATABASE_URL       | postgresql://postgres:password@localhost:5432/placement_tracker_pro?schema=public |
| JWT_SECRET         | replace-with-a-long-secure-random-string                                          |
| JWT_EXPIRES_IN     | 7d                                                                                |
| BCRYPT_SALT_ROUNDS | 12                                                                                |
| CORS_ORIGIN        | http://localhost:5173                                                             |
| GROQ_API_KEY       | gsk_XXXXXXXXXXXXXXXXXXXX (get from console.groq.com)                              |

### 9.3 Start Commands

```
# Backend
cd backend
npm install
npx prisma migrate dev # creates all DB tables
npm run dev            # starts on :5000 with hot reload
```

```
# Frontend (new terminal)
cd frontend
npm install
npm run dev # starts on :5173
```

### 9.4 Database Setup

19. Create PostgreSQL database: CREATE DATABASE placement\_tracker\_pro;
20. Set DATABASE\_URL in backend/.env
21. Run: cd backend && npx prisma migrate dev
22. Optional: seed NeetCode 150 problems via the app UI (DSA Tracker page)

### 9.5 Useful Scripts

| Script       | Command                | What it does                       |
|--------------|------------------------|------------------------------------|
| Backend dev  | npm run dev            | tsx watch — hot reload server      |
| Frontend dev | npm run dev            | Vite dev server with HMR           |
| DB GUI       | npx prisma studio      | Browser UI to view/edit all tables |
| DB migrate   | npx prisma migrate dev | Apply schema changes               |
| Typecheck    | npm run typecheck      | tsc --noEmit without building      |
| Lint         | npm run lint           | ESLint across the whole project    |

## 10. UI Design System

### 10.1 Theme

PlacementTrackerPro uses a professional "bold SaaS" aesthetic — dark sidebar with a light main area (with dark mode support). The color palette is centered on indigo/brand (#6366f1) with surface neutrals and ink darks.

### 10.2 Tailwind Custom Tokens (tailwind.config.ts)

| Token               | Value              | Usage                         |
|---------------------|--------------------|-------------------------------|
| brand.500           | #6366f1            | Primary brand color (indigo)  |
| brand.600           | #4f46e5            | Buttons, active nav, accents  |
| surface.DEFAULT     | #ffffff            | Card backgrounds (light mode) |
| surface.2           | #f8f9fe            | Page background (light mode)  |
| ink.DEFAULT         | #0f0f1b            | Sidebar background            |
| ink.2               | #1e1e30            | Dark card backgrounds         |
| shadow-glow         | brand color glow   | Logo icon, CTAs               |
| shadow-card         | soft shadow        | Dashboard cards               |
| bg-brand-gradient   | indigo gradient    | Buttons, logo, highlights     |
| bg-sidebar-gradient | dark navy gradient | Sidebar background            |

### 10.3 Sidebar Navigation Groups

| Group         | Items                                                         |
|---------------|---------------------------------------------------------------|
| Workspace     | Dashboard, Tasks, Calendar                                    |
| Learning      | DSA Tracker, Core CS, Full Stack, Study History, Analytics    |
| Career        | Placement, Interviews, Mock Interview, Resume Hub, Hackathons |
| AI & Settings | AI Assistant, Settings                                        |

### 10.4 Key Reusable Components

| Component     | Props / Notes                                                         |
|---------------|-----------------------------------------------------------------------|
| DashboardCard | title, description, icon (ReactNode), className — standard card shell |
| StatCard      | label, value, trend — metric display with optional trend arrow        |
| ProgressRing  | value (0-100), label, size — SVG circular progress indicator          |
| ProgressBar   | value, tone (brand/emerald/blue) — horizontal gradient bar            |
| LoadingState  | label — centered spinner card                                         |

| Component         | Props / Notes                                                 |
|-------------------|---------------------------------------------------------------|
| ErrorState        | message — red error card with icon                            |
| NotificationsBell | Reads exam dates and tasks, shows badge count + dropdown      |
| Button            | variant (primary/secondary/ghost/danger), size, loading state |
| Input / Select    | Styled form fields matching the design system                 |

## 11. Live Hackathon Feed

The live feed is a curated in-memory dataset of 10 real events. It does NOT call any external API — the data is static in the opportunityController with deadline math computed at request time.

| Event                        | Organizer             | Type      | Prize              |
|------------------------------|-----------------------|-----------|--------------------|
| Smart India Hackathon 2025   | AICTE / Govt of India | Hackathon | ₹1,00,000+         |
| HackWithInfy 2025            | Infosys               | Hackathon | ₹5,00,000          |
| TCS CodeVita 2025            | TCS                   | Contest   | PPO + ₹2,00,000    |
| TechGig Code Gladiators 2025 | TechGig               | Contest   | ₹5,00,000          |
| Flipkart Grid 6.0            | Flipkart              | Hackathon | ₹7,50,000 + PPO    |
| Amazon HackOn 2025           | Amazon India          | Hackathon | PPO + ₹3,00,000    |
| Microsoft Imagine Cup 2025   | Microsoft             | Hackathon | \$100,000 USD      |
| Google Hash Code 2026        | Google                | Contest   | Trip to Google HQ  |
| ACM ICPC India 2025          | ACM                   | Contest   | World Finals berth |
| HackerEarth Monthly          | HackerEarth           | Contest   | \$500              |

Each event is sorted by days left (ascending). Expired events appear at the bottom. A "closing soon" badge shows when  $\leq 7$  days remain. Users can save events to their personal list.

## 12. Known Limitations & Future Improvements

---

### 12.1 Current Limitations

- No file upload for resumes — ATS scoring requires pasting raw text
- Hackathon feed is static (10 hardcoded events) — not scraped live
- No email verification on signup
- JWT is stored in localStorage (XSS risk); production should use httpOnly cookies
- No rate limiting on AI endpoints — a single user can exhaust Groq quota
- No refresh token mechanism — users need to re-login every 7 days

### 12.2 Planned Improvements

- PDF resume upload + parsing for ATS scoring (stage 4)
- Real-time hackathon scraping from Devpost and Unstop APIs
- Email notifications for upcoming deadlines
- Collaborative features — share DSA progress with study group
- Mobile app (React Native) using the same backend API
- LeetCode integration — auto-sync solved problems
- Admin dashboard for college placement cells

## 13. Full File Structure

```

PlacementTrackerPro/
├── backend/
│   ├── src/
│   │   ├── config/env.ts           # Zod env validation
│   │   ├── controllers/           # 13 controller files
│   │   │   ├── ai.controller.ts
│   │   │   ├── auth.controller.ts
│   │   │   ├── dsa.controller.ts
│   │   │   ├── placement.controller.ts
│   │   │   └── ... (9 more)
│   │   ├── services/             # 13 service files
│   │   │   ├── ai.service.ts      # Groq integration
│   │   │   ├── auth.service.ts
│   │   │   └── ... (11 more)
│   │   ├── routes/               # 13 route files
│   │   │   └── index.ts           # registerRoutes()
│   │   ├── middleware/
│   │   │   ├── auth.middleware.ts # JWT guard
│   │   │   ├── async-handler.ts  # Async error wrapper
│   │   │   ├── error.middleware.ts # Global error handler
│   │   │   └── validate-request.ts # Zod body validation
│   │   ├── utils/
│   │   │   ├── AppError.ts        # Typed error class
│   │   │   ├── jwt.ts             # Sign/verify helpers
│   │   │   └── password.ts        # bcrypt helpers
│   │   ├── data/neetcode150.ts    # 150 curated DSA problems
│   │   ├── app.ts                 # Express app factory
│   │   └── server.ts              # HTTP server entry
│   ├── prisma/schema.prisma      # Full DB schema
│   └── .env                       # Secrets (not committed)
├── frontend/
│   ├── src/
│   │   ├── pages/                # 17 page components
│   │   ├── components/           # Reusable UI components
│   │   │   ├── common/           # Button, Card, Input...
│   │   │   ├── analytics/        # Charts, progress bars
│   │   │   ├── tasks/            # TaskList, TaskForm...
│   │   │   └── ...
│   │   ├── layouts/AppLayout.tsx # Dark sidebar shell
│   │   ├── services/             # 13 API service files
│   │   │   ├── aiService.ts
│   │   │   ├── apiClient.ts      # Axios instance + token
│   │   │   └── ...
│   │   ├── contexts/             # Auth + Toast
│   │   ├── hooks/                # useAuth, useToast...
│   │   ├── types/                # TypeScript interfaces
│   │   └── utils/
│   ├── tailwind.config.ts        # Custom design tokens

```



└ vite.config.ts